CS 431/631 Introduction to Big Data

Big Data Systems: MapReduce and Spark

Some slides are borrowed from Jeff Dean, Sanjay Ghemawat, Dan Weld, et. al.

Lei Yang
Computer Science & Engineering
University of Nevada, Reno

MapReduce: 

initially only Google, later open-sourced in 2015

Hadoop: 

developed based on MapReduce, open-sourced

Background/Motivation:

- Large Scale Data Processing
  - Many tasks: Process lots of data to produce other data
  - Want to use hundreds or thousands of CPUs
    ... but this needs to be easy

- MapReduce provides:
  - Automatic parallelization and distribution
  - Fault-tolerance (off-the-shelf machines)
  - I/O scheduling
  - Status and monitoring

Programming Model

Input & Output: each a set of key/value pairs

Programmer specifies two functions:

map(key, val)
map (in_key, in_value) -> list(out_key, intermediate_value)
- Processes input key/value pair
- Produces set of **intermediate** pairs

reduce(key, val)
reduce (out_key, list(intermediate_value)) -> list(out_value)
- Combines all **intermediate** values for a particular key
- Produces a set of merged output values (usually just one)

Example: Word Count

Input consists of (url, contents) pairs

```
map(key=url, val=contents)
map(String input_key, String input_value):
        // input_key: document name
        // input_value: document contents
        for each word w in input_value:
                EmitIntermediate(w, "1");
                // for each word w in contents, emit (w, “1”)

reduce(key=word, values=uniq_counts)
reduce(String output_key, Iterator intermediate_values):
        // output_key: a word
        // output_values: a list of counts
        int result = 0;
        for each v in intermediate_values:
                result += ParseInt(v); // sum all “1”s in values list
        Emit(AsString(result));      // emit result “(word, sum)”
```

Example: Word Count

This program counts the frequency of words in a text file using MapReduce.

In the **mapper** function, we split each line of text into words and emit each word with a count of 1.

In the **reducer** function, we receive all the counts for each word emitted by the mapper and then sum them up to get the total count for that word.

```
from mrjob.job import MRJob

class MRWordCount(MRJob):

    def mapper(self, _, line):
        words = line.strip().split()
        for word in words:
            yield word.lower(), 1

    def reducer(self, word, counts):
        yield word, sum(counts)

if __name__ == '__main__':
    MRWordCount.run()
```

To run this program, we can save it to a file called **wordcount.py** and then run the following command in the terminal:

```
python wordcount.py input.txt > output.txt
```

Count, Illustrated

map(key=url, val=contents):

For each word *w* in contents, emit (w, "1")

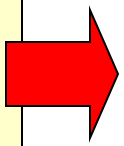
reduce(key=word, values=uniq_counts):

Sum all "1"s in values list

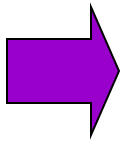
Emit result "(word, sum)"

see bob throw

see spot run



see	1
bob	1
run	1
see	1
spot	1
throw	1

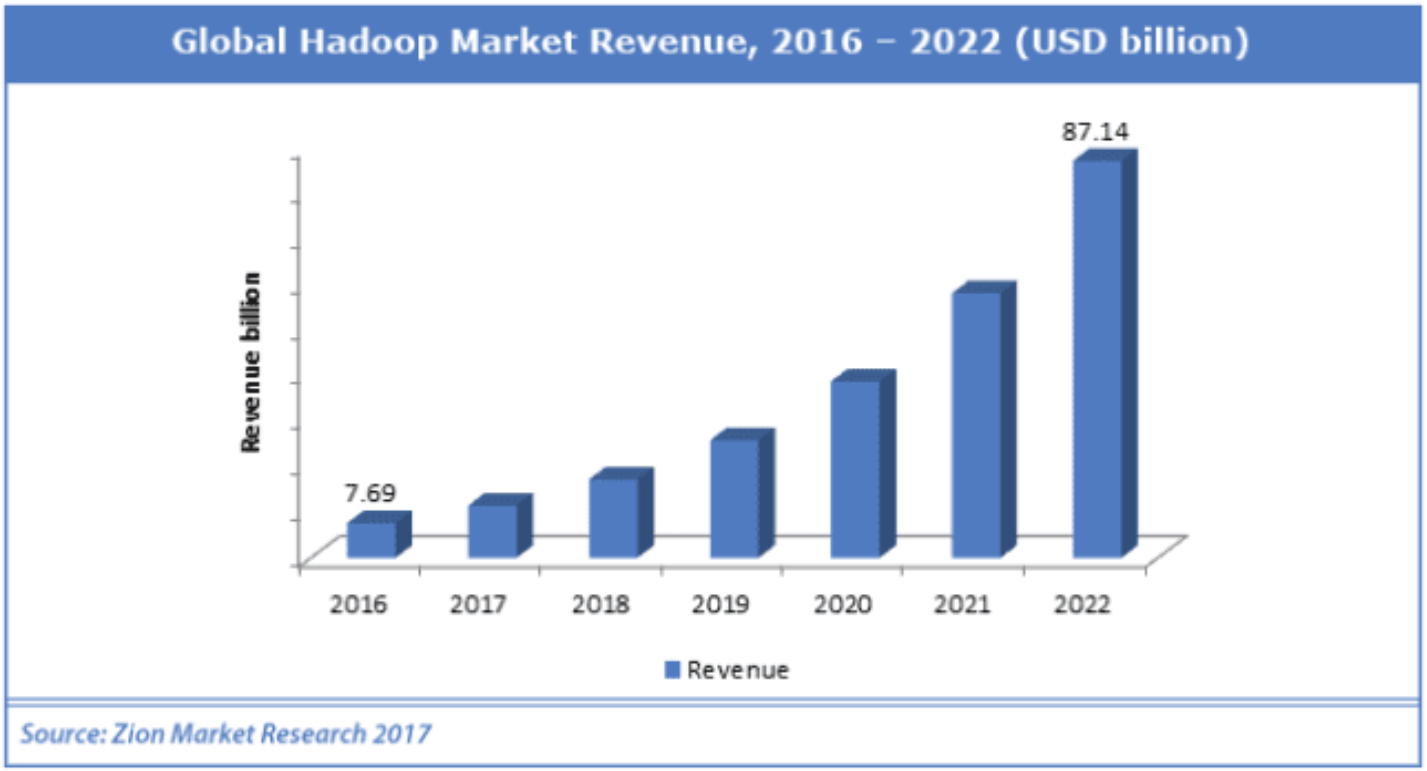


bob	1
run	1
see	2
spot	1
throw	1

Example: Grep

- Input consists of (url+offset, single line)
- map(key=url+offset, val=line):
  - If contents matches regexp, emit (line, "1")
- reduce(key=line, values=uniq_counts):
  - Don't do anything; just emit line

MapReduce/Hadoop is Widely Used



Example uses:

distributed grep	distributed sort	web link-graph reversal
term-vector / host	web access log stats	inverted index construction
document clustering	machine learning	statistical machine translation
...	...	...

Implementation Overview

Typical cluster:

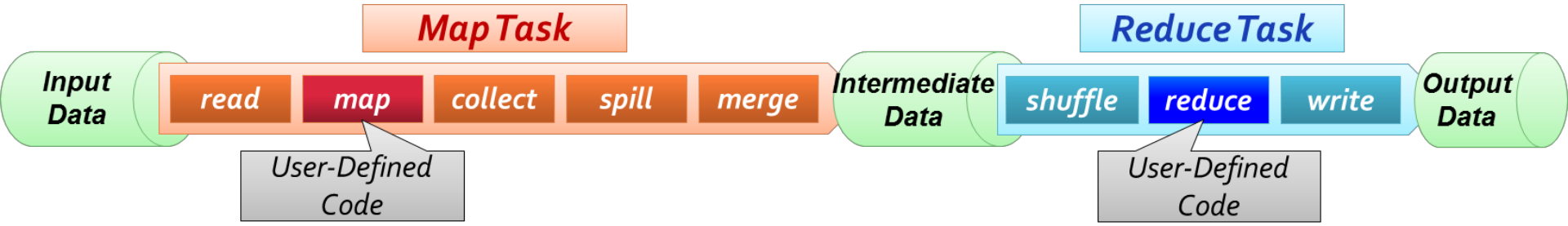
- 100s/1000s of x86 machines, 2-4 GB of memory
- Limited bisection bandwidth
- Storage is on local disks
- GFS/HDFS: distributed file system manages data
- Job scheduling system: jobs made up of tasks, scheduler assigns tasks to machines

Implementation is a C++ library (Java for Hadoop) linked into user programs (any programming language)

Execution

- How is this distributed?
  1. Partition input key/value pairs into chunks, run map() tasks in parallel
  2. After all map()s are complete, consolidate all emitted values for each unique emitted key
  3. Now partition space of output map keys, and run reduce() in parallel

Detailed Execution Steps

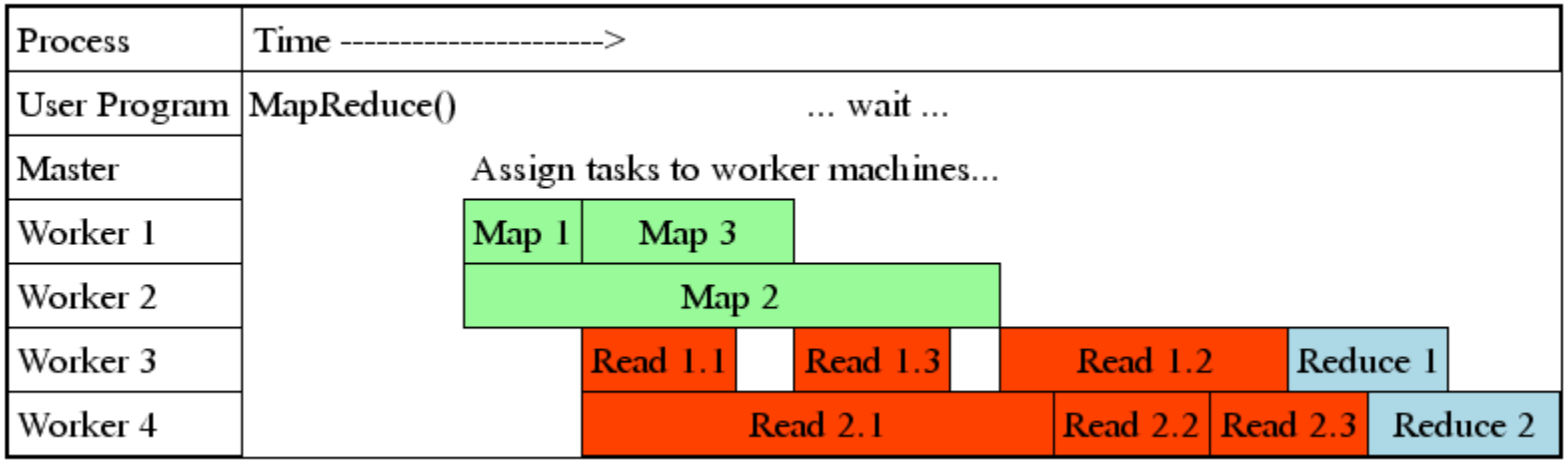


read: reads data from HDFS and turns them into records
map: mappers apply user defined map function on each record
collect: collects output results from map in memory
spill (optional): if intermediate data is larger than memory buffer, spills to disk
merge: merges results into a single file for each reduce task

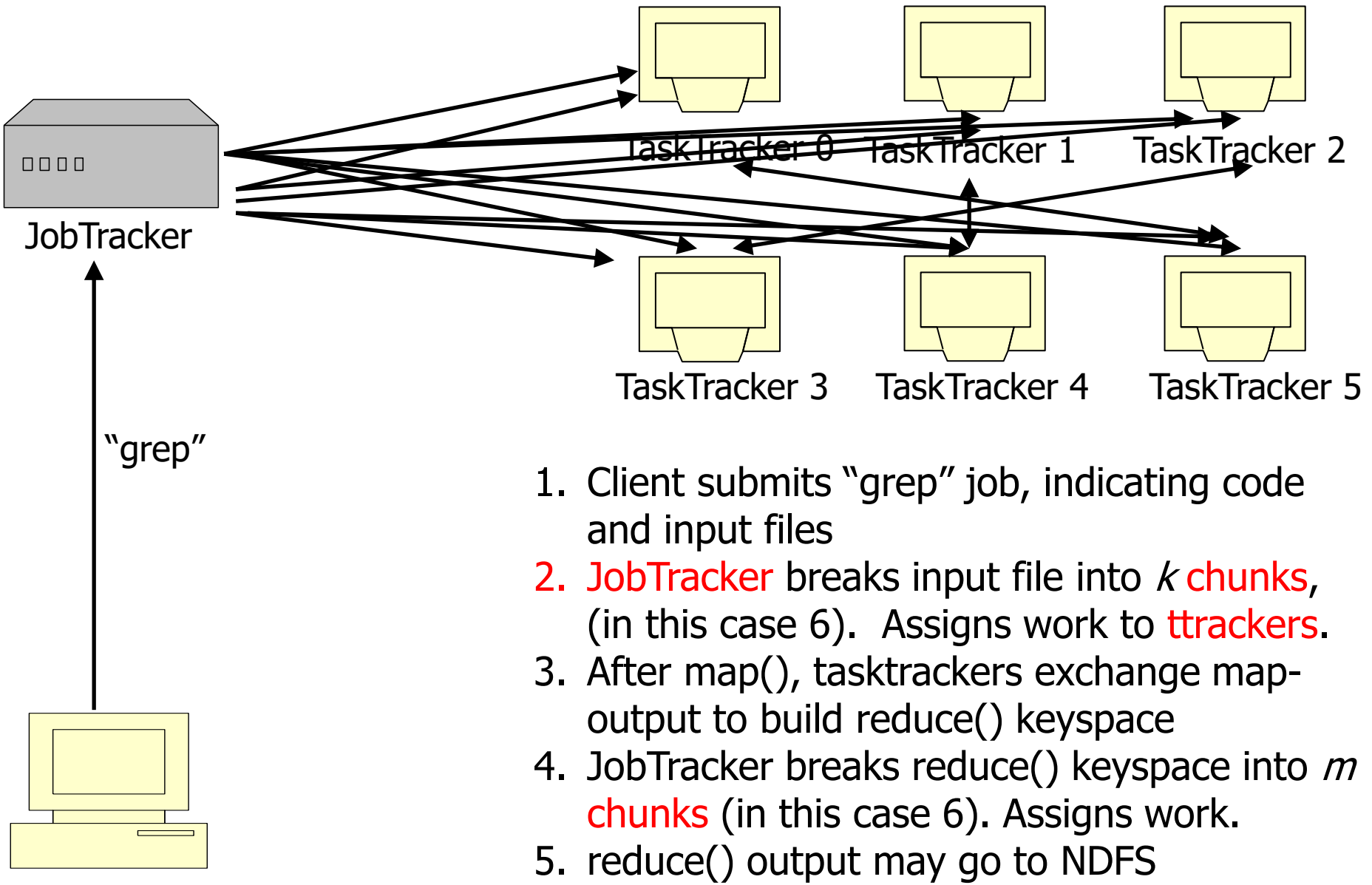
shuffle: reducers fetch, merge, sort intermediate data
reduce: reducers apply user defined reduce function on intermediate data
write: write output to HDFS

Task Granularity & Pipelining

- Fine granularity tasks: map tasks >> machines
  - Minimizes time for fault recovery
  - Can pipeline shuffling with map execution
  - Better dynamic load balancing
- Often use 200,000 map & 5000 reduce tasks
- Running on 2000 machines

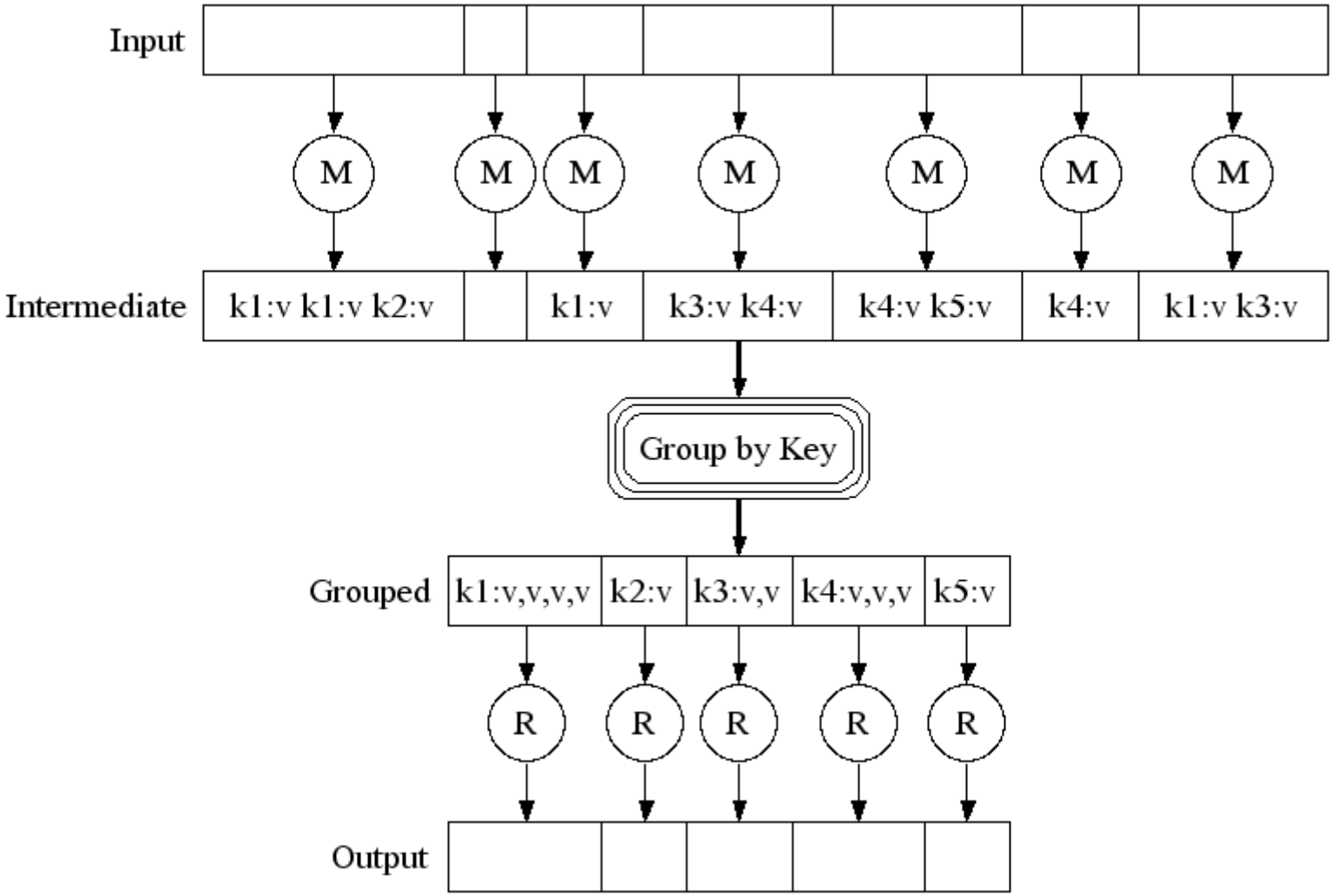


Job Processing



1. Client submits "grep" job, indicating code and input files
2. JobTracker breaks input file into *k* chunks, (in this case 6). Assigns work to ttrackers.
3. After map(), tasktrackers exchange map-output to build reduce() keyspace
4. JobTracker breaks reduce() keyspace into *m* chunks (in this case 6). Assigns work.
5. reduce() output may go to NDFS

Execution


Input
M
M
M
M
M
M
M
Intermediate
k1:v k1:v k2:v
k1:v
k3:v k4:v
k4:v k5:v
k4:v
k1:v k3:v
Group by Key
Grouped
k1:v,v,v,v
k2:v
k3:v,v
k4:v,v,v
k5:v
R
R
R
R
R
Output

Fault Tolerance / Workers

Handled via re-execution

- Detect failure via periodic heartbeats
- Re-execute completed + in-progress *map* tasks
- Re-execute in progress *reduce* tasks
- Task completion committed through master

Master Failure

- Could handle, … ?
- But don't yet
  - (master failure unlikely)

Refinement: Redundant Execution

Slow workers significantly delay completion time

- Other jobs consuming resources on machine
- Bad disks w/ soft errors transfer data slowly
- Weird things: processor caches disabled (!!)

Solution: Near end of phase, spawn backup tasks

- Whichever one finishes first "wins"

Dramatically shortens job completion time

Refinement: Locality Optimization

- Master scheduling policy:
  - Asks GFS for locations of replicas of input file blocks
  - Map tasks typically split into 64MB (GFS block size)
  - Map tasks scheduled so GFS input block replica are on same machine or same rack

- Effect
  - Thousands of machines read input at local disk speed
    - Without this, rack switches limit read rate

Refinement
Skipping Bad Records

- Map/Reduce functions sometimes fail for particular inputs
  - Best solution is to debug & fix
    - Not always possible ~ third-party source libraries
  - On segmentation fault:
    - Send UDP packet to master from signal handler
    - Include sequence number of record being processed
  - If master sees two failures for same record:
    - Next worker is told to skip the record


APACHE
Spark
TM

Overview

- A new programming model (RDD, Resilient Distributed Datasets)
  - parallel/distributed computing
  - in-memory
  - fault-tolerance
- A framework using RDD: Spark

Motivation

- MapReduce greatly simplified "big data" analysis on large, unreliable clusters

- But as soon as it got popular, users wanted more:
  - More **complex**, multi-stage applications (e.g. iterative machine learning & graph processing)
  - More **interactive** ad-hoc queries

Response: *specialized* frameworks for some of these apps (e.g., Pregel for graph processing)

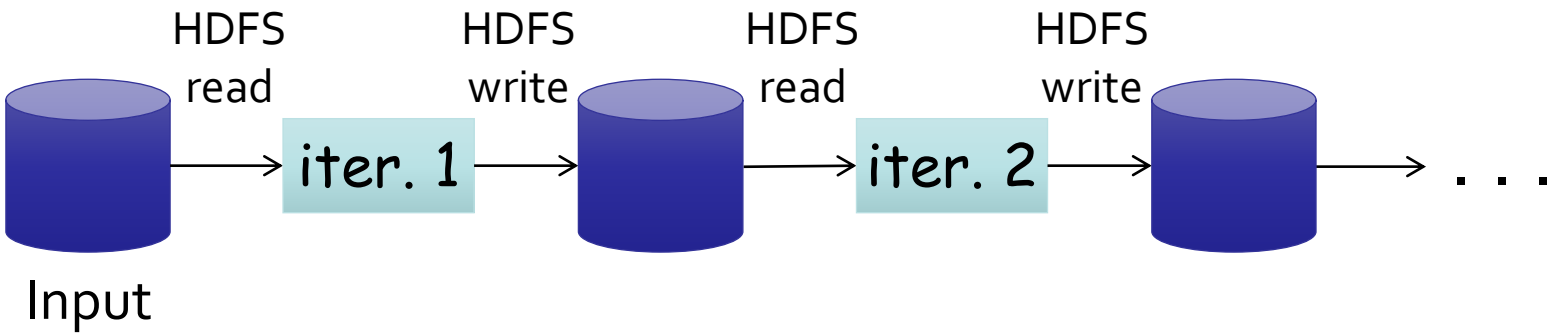
Motivation

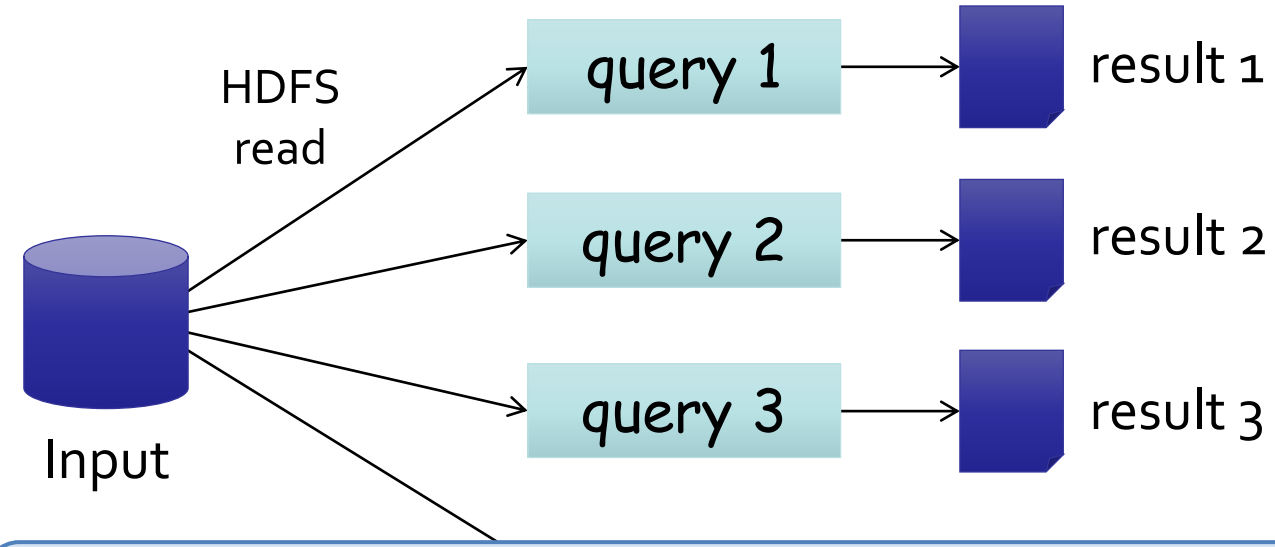
Complex apps and interactive queries both need one thing that MapReduce lacks:

Efficient primitives for **data sharing**

In MapReduce, the only way to share data across jobs is stable storage ➔ slow!

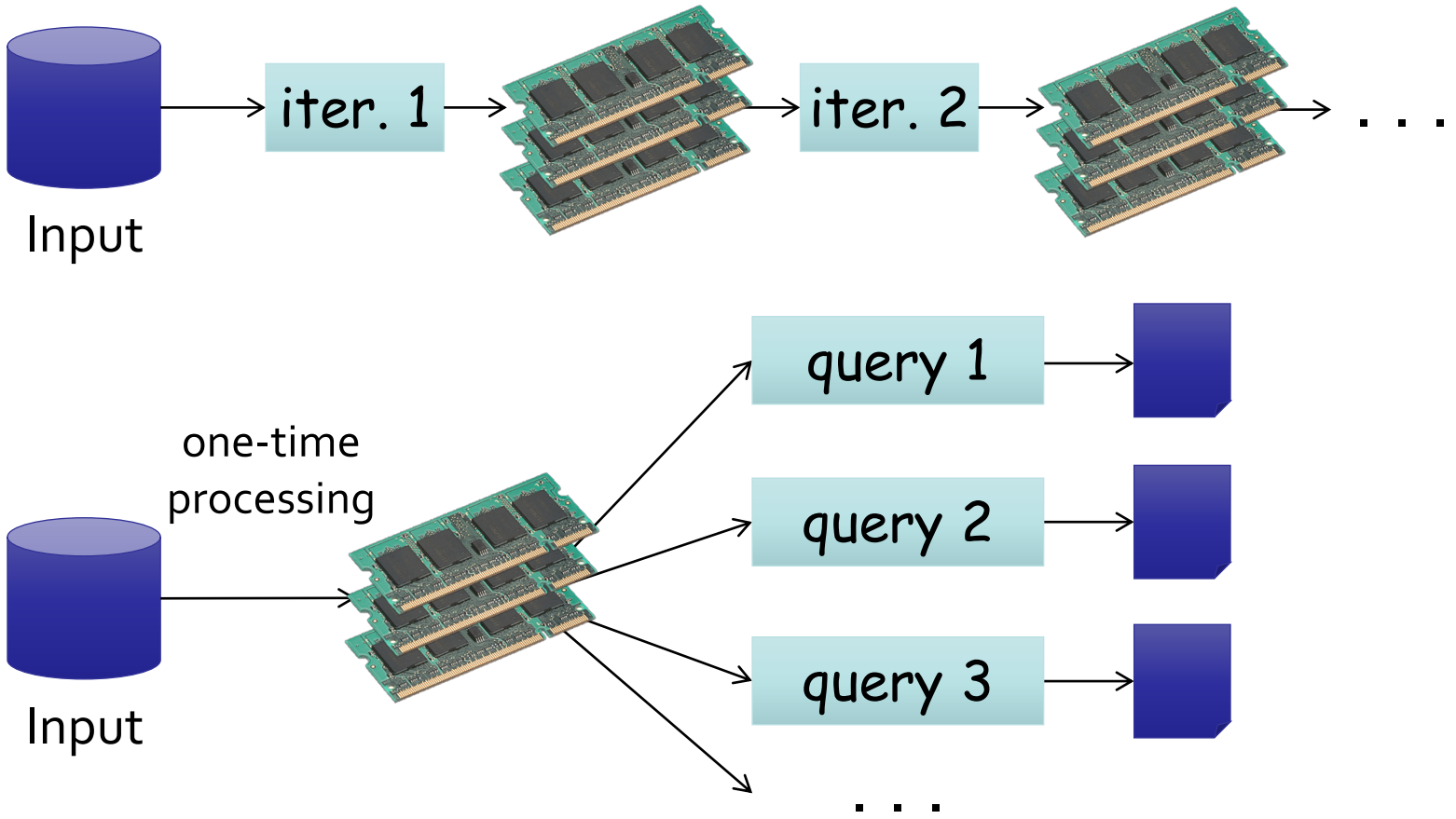
Examples





Slow due to replication and disk I/O, but necessary for fault tolerance

Goal: In-Memory Data Sharing



10-100× faster than network/disk,
but how to get Fault Tolerance?

Challenge

How to design a distributed memory abstraction that is both **fault-tolerant** and **efficient**?

Approach 1: Fine-grained

- E.g., **Piccolo** (Others: RAMCloud; DSM)
- Distributed Shared Table
- Implemented as an in-memory (distributed) key-value store

Fine-Grained: Challenge

- Existing storage abstractions have interfaces based on *fine-grained* updates to mutable state

- Requires replicating data or logs across nodes for fault tolerance
  - Costly for data-intensive apps
  - 10-100x slower than memory write

Observation

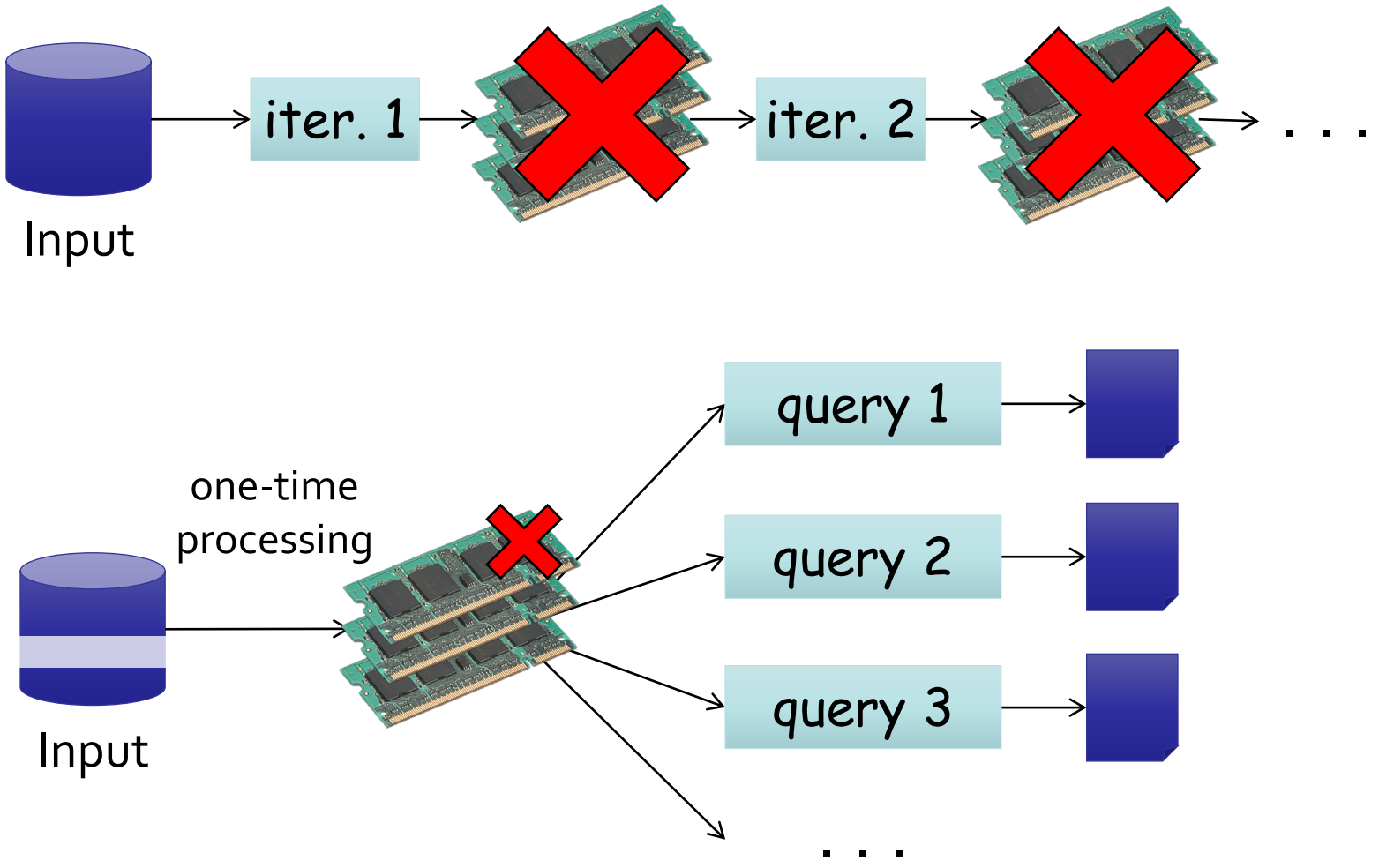
Coarse-grained operation:
In many distributed computing, **same** operation is applied to multiple data items in parallel

Coarse Grained: Resilient Distributed Datasets (RDDs)

- Restricted form of distributed shared memory
  - Immutable, partitioned collections of records
  - Can only be built through *coarse-grained* deterministic transformations (map, filter, join, …)

- Efficient fault recovery using *lineage*
  - lineage: transformations used to build a dataset
  - Record lineage instead of actual data.
  - Recompute lost partitions on failure using lineage
  - No cost if nothing fails

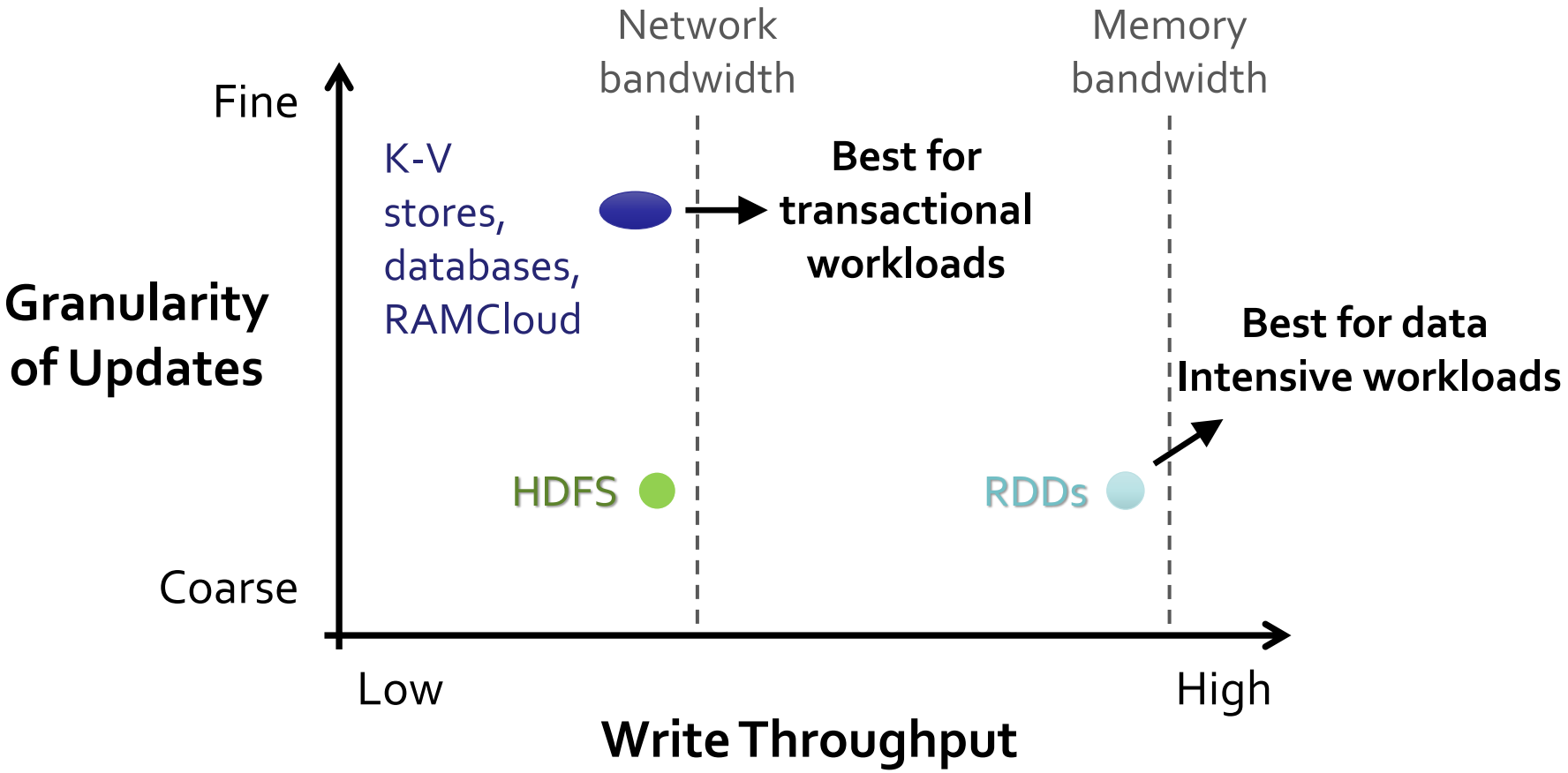
RDD Recovery



Generality of RDDs

- Despite their restrictions, RDDs can express surprisingly many parallel algorithms
  - These naturally *apply the same operation to many items*
- Unify many current programming models
  - *Data flow models*: MapReduce, Dryad, SQL, …
  - *Specialized models* for iterative apps: BSP (Pregel), iterative MapReduce (Haloop), bulk incremental, …
- Support *new apps* that these models don't

Tradeoff Space



Spark Programming Interface

- DryadLINQ-like API in the Scala language
- Usable interactively from Scala interpreter
- Provides:
  - Resilient distributed datasets (RDDs)
  - Operations on RDDs: *transformations* (build new RDDs), *actions* (compute and output results)
  - Control of each RDD's *partitioning* (layout across nodes) and *persistence* (storage in RAM, on disk, etc)

Spark Operations

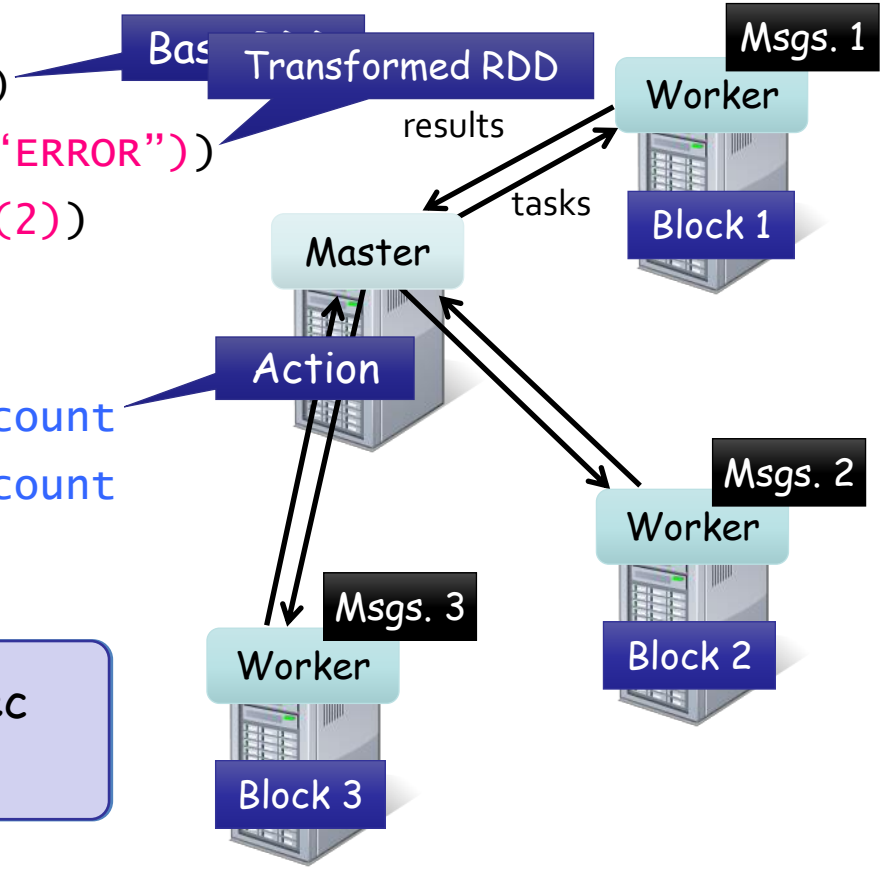
Transformations (define a new RDD)	map filter sample groupByKey reduceByKey sortByKey	flatMap union join cogroup cross mapValues
Actions (return a result to driver program)	collect reduce count save lookupKey	

Example: Log Mining

Load error messages from a log into memory, then interactively search for various patterns

```
lines = spark.textFile("hdfs://...")
errors = lines.filter(_.startsWith("ERROR"))
messages = errors.map(_.split('\t')(2))
messages.persist()

messages.filter(_.contains("foo")).count
messages.filter(_.contains("bar")).count
```

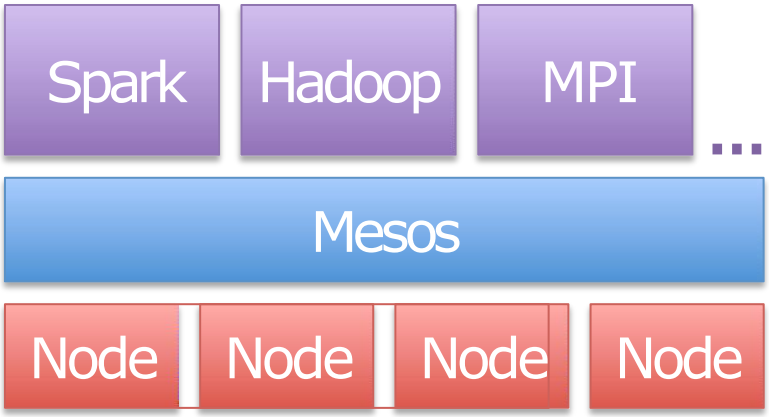


Result: scaled to 1 TB data in 5-7 sec (vs 170 sec for on-disk data)

Implementation

Runs on Mesos [NSDI 11] to share clusters w/ Hadoop

Can read from any Hadoop input source (HDFS, S3, …)



No changes to Scala language or compiler

» Reflection + bytecode analysis to correctly ship code

www.spark.apache.org

WordCount Implementation: Hadoop vs. Spark

```java
public class WordCount {

  public static class TokenizerMapper
       extends Mapper<Object, Text, Text, IntWritable>{

    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    public void map(Object key, Text value, Context context
                    ) throws IOException, InterruptedException {
      StringTokenizer itr = new StringTokenizer(value.toString());
      while (itr.hasMoreTokens()) {
        word.set(itr.nextToken());
        context.write(word, one);
      }
    }
  }

  public static class IntSumReducer
       extends Reducer<Text,IntWritable,Text,IntWritable> {
    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values,
                       Context context
                       ) throws IOException, InterruptedException {
      int sum = 0;
      for (IntWritable val : values) {
        sum += val.get();
      }
      result.set(sum);
      context.write(key, result);
    }
  }

  public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

```scala
val textFile = sc.textFile("hdfs://...")
val counts = textFile.flatMap(line => line.split(" ")).map(word => (word, 1)).reduceByKey(_ + _)
counts.saveAsTextFile("hdfs://...")
```